Will-Wallet

A Non-Custodial, Time-Conditional Protocol for Digital Asset Inheritance

System Architecture, Security Analysis, and Commercialization Model

Yilak Kidane

will-wallet.com

Technical Paper — 2026 Revision

Abstract

Bitcoin's security model — sole custody of an unrecoverable private key — creates an inheritance failure mode absent from traditional financial assets: if a holder dies or becomes incapacitated without transmitting the key, the asset becomes permanently unspendable, verifiably visible on a public ledger yet cryptographically inaccessible to anyone. Existing solutions typically resolve this by reintroducing a custodian — an exchange, bank, or key-escrow service — who holds a working copy of the key, which reproduces exactly the counterparty risk Bitcoin's design was intended to remove.

This paper specifies Will-Wallet, a protocol built entirely from standard, deployed Bitcoin script primitives — M-of-N multisignature outputs, absolute timelocks (CHECKLOCKTIMEVERIFY), and transaction-level nLockTime — that transfers Bitcoin to designated heirs on a chosen future condition without any party other than the original holder ever possessing a working spending key. We show that the protocol admits two structurally distinct release conditions: a deterministic future date or block height, which requires no external party whatsoever and preserves the holder's full revocation rights until the moment of release; and an indeterminate condition such as death, which provably cannot be resolved by the blockchain alone and requires a timing oracle — a role that, we show, can be filled without that party ever holding custodial risk. We further show the protocol composes with a second, independent timelock on each heir's receiving output, producing per-heir vesting schedules (e.g. a trust-fund-style age-of-majority release) layered on top of the inheritance-release condition itself. We give a formal revocability model, a security analysis covering fee-market and script-level failure modes, and a commercialization model for deploying the timing-oracle role through an estate-planning or trust institution.

Contents

1. Introduction

Bitcoin removed the need for a trusted intermediary to hold and move value. That same property — no intermediary can freeze, seize, or reverse a transaction — is what makes death, incapacity, or simple key loss catastrophic rather than merely inconvenient. There is no customer service line to call, no next-of-kin verification process, no court order that restores access. The private key either exists somewhere reachable by the rightful heir, or the funds are permanently lost.

Industry practice today addresses this in one of three ways, each with a distinct failure mode: (i) informal key sharing — writing the key down and hoping it is found, understood, and not stolen; (ii) custodial inheritance services, which hold a working key or seed on the client's behalf, reintroducing single-point-of-failure custodial risk (the custodian can be hacked, coerced, go insolvent, or simply make an error); and (iii) legal instruments (wills, trusts) that reference the asset but cannot themselves execute a Bitcoin transaction, leaving execution to an executor who may have no technical means of acting on it.

This paper specifies a fourth approach: encode the inheritance plan directly into pre-authorized, script-enforced Bitcoin transactions such that (a) no party other than the original holder ever possesses a key capable of spending the funds during the holder's control period, (b) the release condition — a date, or a real-world event — is enforced either by Bitcoin's own consensus rules or by a party whose role is strictly informational, and (c) the holder retains full ability to revise or revoke the plan for as long as it remains technically reversible, with that boundary made explicit rather than implicit.

2. Background and Related Work

2.1 Multisignature Outputs

A Pay-to-Script-Hash (P2SH) or Pay-to-Witness-Script-Hash (P2WSH) multisignature output specifies M and a set of N public keys; the output can be spent by a transaction bearing valid signatures from any M of the N corresponding private keys, in effect since BIP-11/BIP-16. Standard configurations (2-of-2, 2-of-3, 3-of-5) are used in exchange cold storage and joint accounts. Will-Wallet uses the general M-of-N construction with an asymmetric key distribution across a single family: the holder retains more keys than any single heir, altering the threshold's meaning from “shared custody” to “unilateral holder control with heir participation required only for cooperative release.”

2.2 Timelocks: Two Independent Mechanisms

Bitcoin provides two unrelated ways to make funds unspendable until a future point, and conflating them is the single most common design error in “crypto inheritance” proposals, including an earlier version of this project's own 2018 paper.

Mechanism	Where it lives	What it restricts	Who is bound by it
nLockTime(transaction field)	In the transaction itself	When this specific, already-built transaction becomes valid to broadcast/mine	Only that one transaction — the underlying coins can still be spent by any other valid transaction before that time

Mechanism	Where it lives	What it restricts	Who is bound by it
CHECKLOCKTIMEVERIFY(BIP-65, script opcode)	In the output's script (the address)	When the coins sitting at this address can be spent at all, by any transaction	Every future transaction attempting to spend that output — no signature combination bypasses it

This distinction is the load-bearing idea of the entire protocol: a transaction-level lock restricts one piece of paper; a script-level lock restricts a vault. The two can be composed, as §5 develops.

2.3 Related Approaches

- Custodial inheritance services (various exchanges, dedicated “crypto will” companies): hold a working key or shard; convenient, but the provider becomes a permanent, ongoing custodial risk and a legal money-transmission question.
- Shamir's Secret Sharing of a single key across trustees: removes single-point custody of the whole key, but shares are static — they cannot express a date condition, cannot be selectively revoked per-heir, and reconstruction still hands a live spending key to whoever assembles the threshold.
- Dead-man's-switch “reveal services” (e.g. scheduled email release of a key or seed phrase): the released artifact is the key itself, so the service (or anyone who intercepts the release) gains full custody the instant it fires — a strictly worse trust profile than releasing an inert half-signed transaction, developed in §4.

3. Threat Model and Trust Assumptions

3.1 Parties

Party	Holds	Trusted for
Holder (“Sender”)	≥ threshold−(N−1) keys — e.g. 2 of 3	Constructing correct distribution transactions; nothing else is assumed
Heir(s)	The remaining key(s), possibly one key shared across all heirs	Nothing — assumed potentially adversarial toward each other (see §6.2)
Timing party (optional)	A half-signed transaction and/or a monitoring responsibility	Correct timing only — explicitly not trusted with funds

3.2 Adversary Goals Considered

1. Early release — an heir or the timing party obtains spendable funds before the intended date.
2. Collusion — a subset of heirs combine signatures to divert funds without the holder's authorization.
3. Griefing — a party broadcasts a stale or superseded transaction to force an unwanted outcome.

4. Timing-party compromise — the party responsible for noticing the release condition is bribed, coerced, or negligent.

5. Holder key loss — the holder loses their own keys prior to the release event, before ever exercising revocation.

Each is addressed directly in §8.

4. System Architecture

Figure 1 — Key Custody & Trust Topology

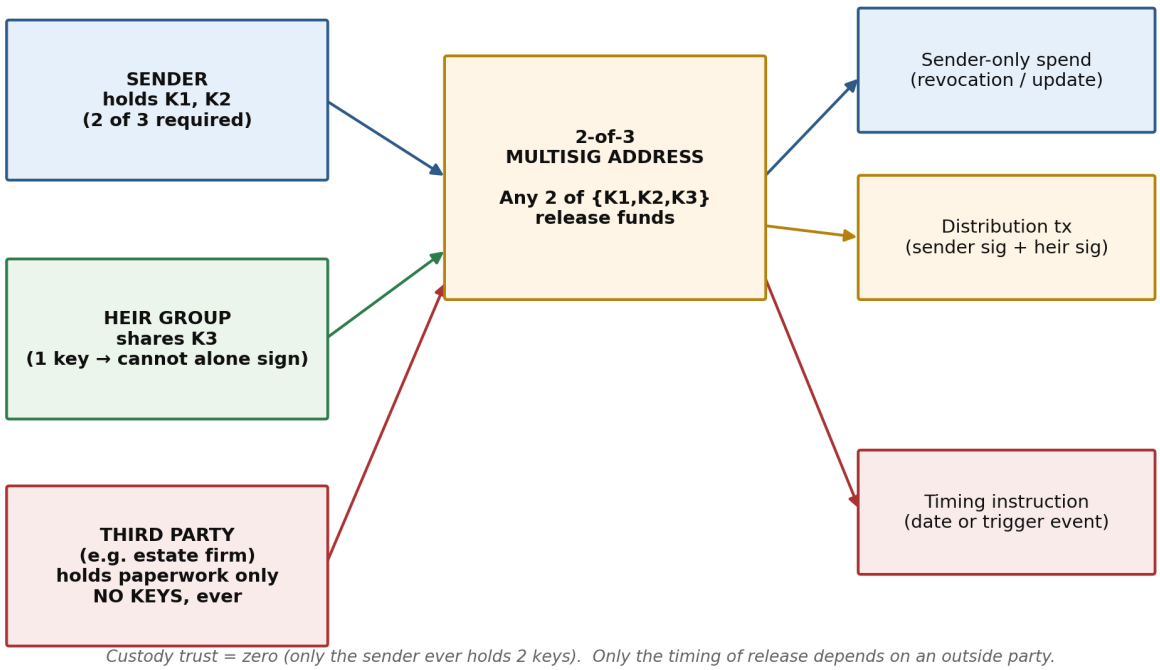

Figure 1. Key custody is confined to the holder and the heir group; the timing party never holds a key.

4.1 Key Structure

The base configuration is a 2-of-3 P2WSH multisignature address with public keys K1, K2 (both controlled by the holder, ideally on separate signing devices for operational-security redundancy) and K3. For a single heir, K3 belongs to that heir. For multiple heirs sharing one disbursement, K3 is distributed identically to every heir — a deliberate design choice analyzed in §6.2, chosen specifically because a shared single key cannot be partially combined by any subset of heirs to reach the 2-of-3 threshold without the holder.

4.2 The Half-Signed Transaction Primitive

The holder constructs the intended distribution transaction — specifying every output address and amount — and signs it with exactly one of K1/K2. A transaction bearing one of two required signatures is not a valid, minedable Bitcoin transaction under any circumstance; it is inert data. It cannot be broadcast, cannot be relayed by any node's mempool policy, and grants its holder no ability to affect the referenced UTXO. This is the property that allows it to be handed to a party who is not fully trusted (§7) without introducing custodial risk: possession of a half-signed transaction is equivalent, in security terms, to possession of a photograph of a signed check missing its second signature.

4.3 Distribution and Completion

1. Holder funds the 2-of-3 address.
2. Holder constructs the distribution transaction (one output per heir) and signs with K1 (or K2).
3. The half-signed transaction is distributed to heirs, gated by one of the two release conditions in §5.
4. Any heir holding K3 (or their individual key, in the per-heir-key variant) adds the second signature and broadcasts.
5. The transaction confirms; funds move to the output addresses specified in step 2 — which may themselves carry a further script-level timelock (§5.3).

5. Temporal Model: Two Independent Timelocks

The protocol composes two timelocks that operate at different layers and answer different questions. Layer 1 (transaction-level, §2.2) governs when the distribution transaction itself may be broadcast — i.e. when the estate is distributed at all. Layer 2 (script-level CLTV on each output address) governs when a given heir may spend what they were already given — i.e. a vesting schedule on top of the distribution, exactly analogous to a trust fund that releases principal to a beneficiary only at age of majority, independent of when the trust itself was funded.

Figure 2 — Two-Stage Temporal Release (per-heir vesting)

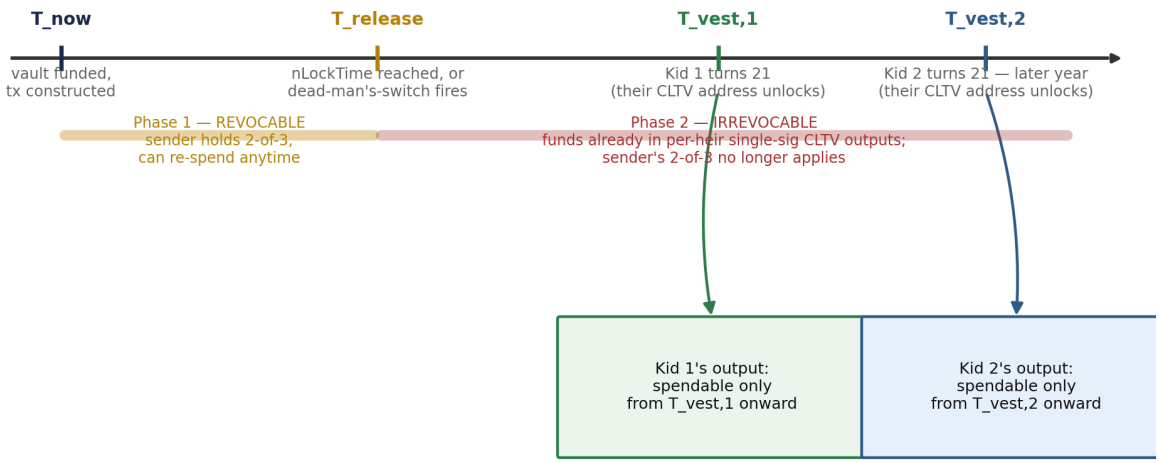

Figure 2. Distribution timing (Layer 1) is independent of — and typically earlier than — each heir's individual vesting date (Layer 2).

5.1 Layer 1 — Distribution Timing

Two mutually exclusive modes, selected per plan:

- Known-date mode: the distribution transaction's nLockTime field is set to the target date or block height. No relay or mining node will accept it before that point, under any condition (BIP-113 clarifies this against median-time-past). The holder retains full revocation rights the entire time (§6).

- Unknown-date mode (§7): no meaningful nLockTime exists for “whenever I die.” Distribution timing must be resolved by an external, low-trust process.

5.2 Prerequisite: Sequence Number

nLockTime is only enforced if at least one transaction input carries a sequence number below 0xFFFFFFFF (BIP-68 non-final signaling). A transaction built with default-max sequence numbers will be treated as final and relayable immediately regardless of its nLockTime field — a common implementation error that silently defeats the entire known-date mechanism. Any implementation of this protocol must assert non-final sequence numbers on every input as a hard precondition, and this assertion should be part of automated test coverage before any production release.

5.3 Layer 2 — Per-Heir Vesting via CHECKLOCKTIMEVERIFY

Each heir's output in the distribution transaction can pay not to a plain address but to a CLTV-gated script of the form “<T_vest,i> CHECKLOCKTIMEVERIFY DROP <heir pubkey> CHECKSIG,” where T_vest,i is set independently per heir — for example, each child's 21st birthday, which in general falls on a different calendar date for each sibling. Because this restriction is compiled into the output's script rather than into any one transaction, it binds every future spending attempt from that address; unlike Layer 1, there is no equivalent of “any other transaction can still move it,” because no other valid spending path exists once the coins are locked to that script.

Composability rule

- Layer 1 controls when the estate is distributed. Layer 2 controls when each heir can spend what they were given. They are independent variables: a plan can distribute immediately on death (Layer 1 via dead-man's-switch) while still vesting each minor heir's share at their own age of majority (Layer 2 via per-heir CLTV) — replicating a trust fund's economic behavior entirely in script, with no trustee required to enforce the vesting schedule.

5.4 Time Basis

Both nLockTime and CLTV interpret their argument as either a block height or a Unix timestamp, distinguished by whether the value is below or above 500,000,000. Calendar-accurate release dates (birthdays, fixed anniversaries) should use the timestamp form; block-height locks drift against calendar time because block production is a stochastic ~10-minute average, not a clock.

5.5 Per-Heir Installment Scheduling

The single-output-per-heir construction in §5.3 generalizes directly to multiple outputs per heir within the same distribution transaction. A Bitcoin transaction is not limited to one output per recipient; the holder may instead give a given heir N separate outputs, each paying to that heir's own public key, each wrapped in its own CLTV script with an independently chosen unlock date $T_1 < T_2 < \ldots < T_N$ and an independently chosen amount. All N outputs are constructed and signed once, inside the same half-signed transaction described in §4.2 — no additional signing event, no additional cooperation from the heir, and no change to the underlying primitive.

This turns a single vesting date into a full installment schedule: for example, a minor heir reaching age 21 in 2029 could be given X at 2029, Y at 2030, Z at 2031, and so on, each amount sitting in its own output and becoming spendable independently the moment its own date arrives, with no requirement to claim earlier installments first or in order. The same construction applies equally to structured settlements or

insurance-style payout schedules, since nothing in it is specific to age-based vesting — only the choice of dates and amounts changes.

Cost consideration, not a limitation

- Every additional installment is an additional transaction output, which grows the transaction's size and one-time construction cost. This is negligible for a handful of installments (e.g. 5–10 annual payouts) but becomes unwieldy at very fine granularity (e.g. 240 monthly outputs over 20 years) — at that scale, several smaller distribution transactions constructed over time are a more practical implementation than one very large one.

6. Revocation and Update Semantics

Figure 3 — Revocability State Machine

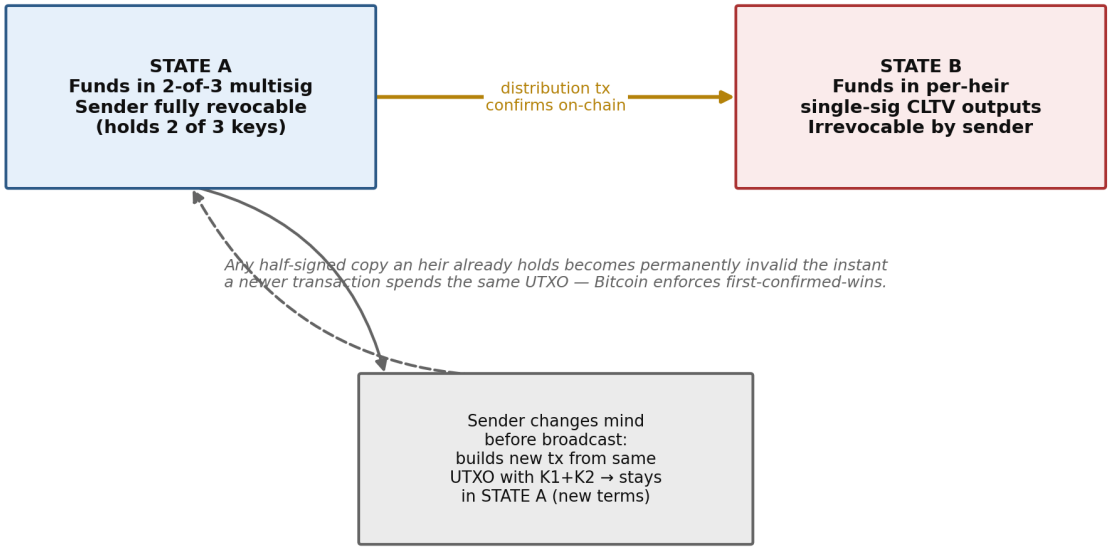

Figure 3. The system has exactly one irreversible transition: confirmation of the distribution transaction.

6.1 Unilateral Update

Because the holder alone controls 2 of the 3 keys, the holder alone already meets the spending threshold and requires no cooperation — from heirs or from any timing party — to change the plan. To alter beneficiaries or amounts, the holder constructs an entirely new transaction spending the same UTXO with K1+K2 and broadcasts it directly. On confirmation, the original UTXO is spent; every previously distributed half-signed transaction referencing it becomes permanently invalid, rejected by every node as a double-spend, regardless of how many copies exist or who holds them. No key rotation, re-keying, or heir notification is required for this to take effect — it is a direct consequence of Bitcoin's first-confirmed-wins consensus rule, not an additional feature the protocol must implement.

6.2 Collusion Resistance Under a Shared Heir Key

A naive multi-heir generalization — a 2-of-(2+N) multisig with a distinct key per heir — permits any two heirs to jointly reach the signing threshold and construct an unauthorized distribution without the holder, since two heir signatures alone satisfy "any 2 of N+2." This is a genuine flaw in that construction, not a hypothetical one: with 8 heirs it admits 28 unauthorized signing pairs. The fix used in this protocol is to issue the identical key K3 to every heir rather than a distinct key each. Because every heir holds the same single key, any number of colluding heirs can produce at most one distinct signature between them — never two — so the 2-of-3 threshold can never be met without a holder signature under any coalition size. This preserves the flexibility of an arbitrary number of heirs while reducing the collusion surface to zero, at the cost of losing the ability to disable one heir's copy of K3 independently (addressed in §6.1: disabling an heir is achieved by re-spending the UTXO, not by revoking their key).

6.3 The One Irreversible Transition

The protocol has exactly one point of no return: confirmation of the distribution transaction. Before it, the holder's 2-of-3 control is absolute and unilateral. After it, funds sit in per-heir output scripts (single-key, possibly CLTV-gated) over which the holder has no further authority whatsoever — not partial, not conditional. Any product built on this protocol should surface this boundary explicitly to the holder (e.g. a confirmation step reading "this action is irreversible") rather than leaving it as an implicit consequence of the script design.

7. The Irreducible Case: Unknown-Date Release

Layer 1's known-date mode removes external trust entirely. It cannot, by construction, address the release condition most people actually want: "when I die," an event whose timing is not a computable function of anything observable on a blockchain. No script, oracle protocol, or cryptographic construction can resolve this without an input from outside the system — this is not a limitation of Bitcoin script specifically but a structural fact about the class of "detect a real-world event" problems, the same reason every legal will names a human executor rather than a self-executing clause.

7.1 Minimal-Trust Timing Oracle

The critical design property is that the timing party's role can be reduced to detection and delivery only, never custody. Two implementations:

- Dead-man's-switch: the holder performs periodic check-ins (e.g. every 90 days) with a monitoring service; failure to check in triggers release of the already-inert half-signed transaction to the heirs.
- Institutional executor: an estate-planning firm already performing death/incapacity determination as part of normal probate practice releases the same artifact as part of its existing workflow (developed further in §10).

In both cases the party's compromise, bribery, or negligence can cause only a timing error (early or late release), never theft — the half-signed transaction remains unspendable without an heir's signature regardless of who leaks or mishandles it. This is the precise, narrow sense in which the phrase "no trusted third party" should be understood for this protocol: it is accurate for custody, and inapplicable to the unknown-date branch's timing, which is an unavoidable, separate trust dimension.

8. Security Analysis and Limitations

Issue	Impact	Mitigation
Sequence-number omission defeats nLockTime (§5.2)	Known-date distribution becomes broadcastable immediately	Hard-code non-final sequence numbers; test for this explicitly
Stale fee rate on long-held pre-signed tx	May fail to confirm promptly at broadcast time	Reserve a CPFP-anchor output on every constructed transaction (dust output any holder can spend to bump fees)
Block-height vs. timestamp ambiguity	A miscoded lock value silently means the wrong thing (§2.2/5.4)	Always use the timestamp form for calendar-based conditions; validate range at construction time
Holder key loss before revocation is ever exercised	Plan becomes frozen at its last-set terms; not itself dangerous, but forecloses future changes	Standard hardware-wallet backup practice for K1/K2; out of scope for the protocol itself
Single-key CLTV output has no backup path (§5.3)	An heir losing their key after Layer 1 release loses that share permanently	Optionally use a 1-of-2 (heir key + a neutral recovery key held by the timing institution, itself never sufficient alone once combined with a threshold design) for high-value shares
Distribution-transaction race (§6.1)	If the holder revokes at nearly the same instant an heir broadcasts, outcome depends on which is mined first	Not a security flaw — an inherent property of any unilateral on-chain revocation — but should be disclosed to holders as a small window of nondeterminism

8.1 Legal and Tax Positioning

This protocol determines transaction validity and timing; it does not determine legal title, does not substitute for a will or trust instrument in any jurisdiction, and creates no binding assurance regarding tax treatment of the transfer. Any production deployment should treat this as infrastructure sitting alongside conventional estate-planning instruments, not a replacement for them, and should avoid making tax-characterization claims without qualified legal review.

9. Trust Minimization Taxonomy

Precision matters here because “trustless” is frequently used as a single undifferentiated claim. This protocol should be described along three independent axes:

Trust dimension	Who could violate it	Status in this protocol
Custody (can someone move my funds without authorization?)	Any key holder outside the intended threshold	Eliminated — no party other than the holder ever holds a working 2-of-3 combination
Distribution timing, known-date mode	N/A — enforced by consensus rules	Eliminated — no party required

Trust dimension	Who could violate it	Status in this protocol
Distribution timing, unknown-date mode	The timing oracle (delay, early trigger, negligence)	Irreducible — minimized to detection/delivery only, never custody (§7.1)
Vesting timing (Layer 2)	N/A — enforced by consensus rules once funded	Eliminated once the CLTV output is funded

10. Business Model

The commercial opportunity sits precisely at the one trust dimension the protocol cannot eliminate: unknown-date timing. An estate-planning or trust institution is structurally suited to this role because determining death or incapacity, and acting on a client's prior instructions at that point, is already its core function — the protocol adds one new artifact to an existing workflow rather than requiring a new one.

10.1 Division of Responsibility

Function	Owned by
Key generation, transaction construction, script correctness	Protocol / software provider (Will-Wallet)
Custody of any key	Never held by either party — holder and heirs only
Death/incapacity determination, check-in monitoring, release trigger	Estate/trust institution
Client relationship, existing estate plan integration, compliance	Estate/trust institution
Software interface, ongoing protocol maintenance	Protocol / software provider

10.2 Revenue Structure

1. Per-vault setup fee — one-time charge for key generation and transaction construction at plan creation.

2. Annual monitoring subscription — recurring fee scoped only to unknown-date plans, which are the only ones requiring ongoing institutional effort; known-date plans carry no recurring cost and should not be billed as if they did.

3. Multi-heir / multi-vesting tier — incremental fee for additional heirs or independently scheduled Layer-2 vesting dates.

4. Protocol licensing / white-label fee — the institution offers the service under its own brand on licensed Will-Wallet infrastructure.

5. Referral share on adjacent legal services — wills, trusts, and broader estate plans originated alongside a vault.

10.3 Why This Is a Favorable Position for the Institution

- No custody of client funds at any point — materially different regulatory posture than a crypto custodian or money transmitter.
- Fits an existing operational competency (death/incapacity determination) rather than requiring a new one.
- Addresses an unmet need among existing high-net-worth clients already holding Bitcoin with no adequate succession plan.
- Differentiates from custodial “crypto inheritance” competitors on the exact dimension security-conscious clients care most about.

11. Future Work

- Migration from legacy P2SH/P2WSH multisig scripts to Taproot with MuSig2 key aggregation — smaller, cheaper, and more private transactions that do not reveal the M-of-N structure on-chain until spent.
- Miniscript-based descriptor wallets for auditable, machine-verifiable script correctness prior to funding.
- PSBT (BIP-174) as the standard interchange format for the half-signed transaction, for compatibility with existing hardware wallets.
- OP_CHECKTEMPLATEVERIFY (BIP-119, if activated) as a potential alternative to pre-signed transactions for expressing distribution templates without holding a live signature at all.
- Multi-asset extension beyond Bitcoin to other UTXO-model assets sharing the same script primitives.

12. Conclusion

Will-Wallet demonstrates that Bitcoin inheritance can be solved using only standard, already-deployed script primitives, with no party other than the original holder ever gaining custodial access to the funds. The protocol cleanly separates two questions that prior proposals conflate — revocability of the distribution plan, and vesting of what each heir eventually receives — and shows both can be enforced independently through nLockTime and CLTV respectively. The one trust dimension that remains, timing an unknown future event such as death, is not a flaw in the design but a structural property of the problem itself, and is minimized to a detection-and-delivery role well suited to an existing estate-planning institution rather than a new custodial intermediary.

Appendix A — Bitcoin Primitives Reference

Reference	Primitive
BIP-11 / BIP-16	Multisignature (P2SH) output construction
BIP-65	OP_CHECKLOCKTIMEVERIFY — script-level absolute timelock
BIP-68 / BIP-112	Relative timelocks (sequence numbers); nLockTime enforcement precondition
BIP-113	Median-time-past semantics for timestamp-based locks
BIP-125	Replace-by-Fee (RBF) opt-in fee bumping

Reference	Primitive
BIP-174	Partially Signed Bitcoin Transaction (PSBT) interchange format
BIP-119 (proposed)	OP_CHECKTEMPLATEVERIFY — covenant-style transaction templating